



산술 연산

산술 연산은 덧셈, 뺄셈, 곱셈, 나눗셈과 같은 사칙 연산을 하는 것을 말합니다. 산술 연산은 정수 및 실수에 대하여 수행되며 연산의 우선순위는 우리가 배운 수학에서 적용되는 우선순위와 비슷합니다.

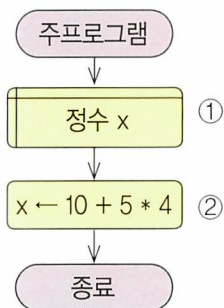
일반적으로 프로그래밍 언어에 적용되는 산술 연산자의 우선순위는 [표 2-1]과 같습니다.

우선 순위	연산자
1	-(음수)
2	*(곱셈), /(나눗셈)
3	+(덧셈), -(뺄셈)

[표 2-1] 산술 연산자

산술 연산을 수행하는 알고리즘은 다음과 같습니다. 예를 들어 $10 + 5 * 4$ 을 연산해 변수 x 에 저장해 봅니다. 이때, 변수 x 에는 30이 저장됩니다.

순서도 예



① 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 x 는 정수를 저장하기 위한 변수입니다.

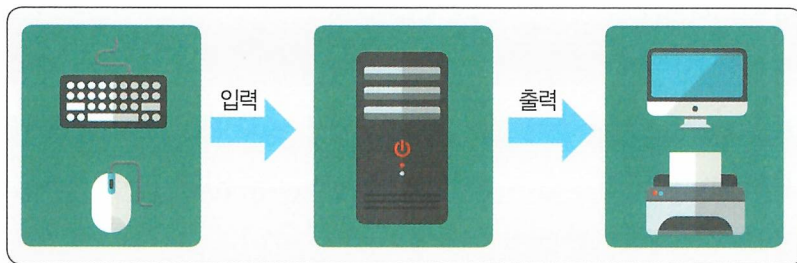
② 변수에 식을 연산한 결과를 저장합니다.

- $5 * 4$ 를 먼저 연산한 후 10을 더한 결과를 변수 x 에 저장합니다.



입 · 출력

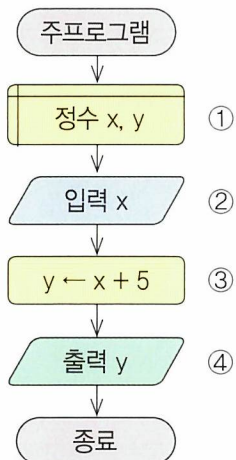
프로그램 특성 중의 하나는 데이터를 입력 받아 처리하여 출력하는 것입니다. 즉, 프로그램을 실행하여 외부로부터 데이터를 입력 받을 수 있고 이를 처리한 결과를 출력할 수 있습니다. 입력과 출력을 하는 명령은 프로그래밍 언어마다 다르며 원하는 형태로 입 · 출력을 하기 위해서는 명령의 세부 사항을 잘 살펴보아야 합니다.



[그림 2-1] 컴퓨터 시스템과 입 · 출력

입출력을 수행하는 알고리즘의 예는 다음과 같습니다.

순서도 예



- ① 이 알고리즘에서 필요한 변수는 다음과 같습니다.
- 변수 x, y는 정수를 저장하기 위한 변수들입니다.
- ② 데이터를 입력 받습니다.
- 정수를 입력 받아 변수 x에 저장합니다.
- ③ 변수에 식을 연산한 결과를 저장합니다.
- 변수 x에 저장되어 있는 값과 5를 더하여 변수 y에 저장합니다.
- ④ 결과를 출력합니다.
- 변수 y에 저장되어 있는 값을 외부에 표시합니다.

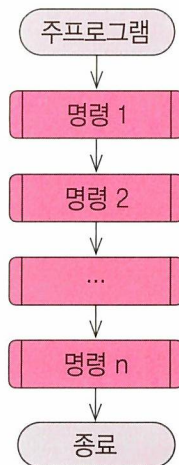


순차 구조

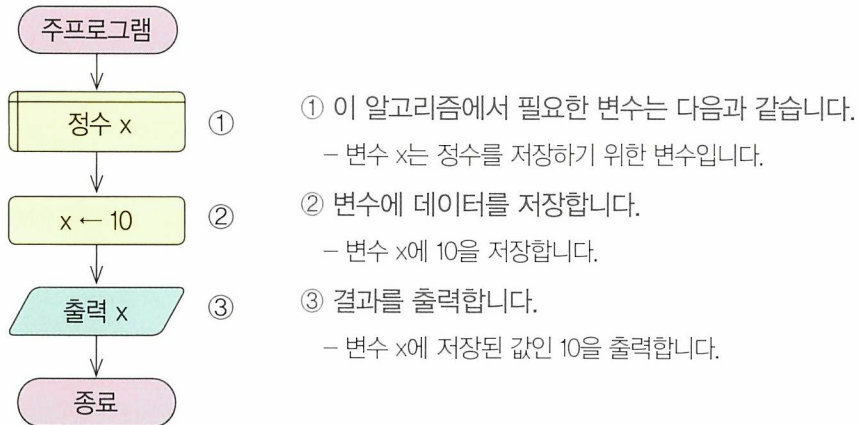
구조화 프로그래밍은 3개의 구조를 기반으로 하여 프로그램을 작성하는 것을 말합니다. 구조화된 프로그램은 설계의 효율성을 높여주며 프로그램을 읽기 좋게 해 줍니다. '프로그램이 읽기 좋다'는 말은 사람들이 이해하기 쉽다는 말이 되며 이는 오류를 발견하거나 개선하는데 효과적이라는 말과 같습니다. 따라서 읽기 좋은 프로그램을 작성하는 것은 소프트웨어 개발에 있어서 아주 중요한 요소 중의 하나입니다.

순차 구조는 프로그램이 표현된 순서대로 실행되는 것으로, 구조화 프로그램의 가장 기본적인 구조입니다. 별도의 형태를 가지지 않은 경우가 많으며 아무런 언급이 없는 경우에 알고리즘 혹은 프로그램은 표현된 순서대로 실행됩니다.

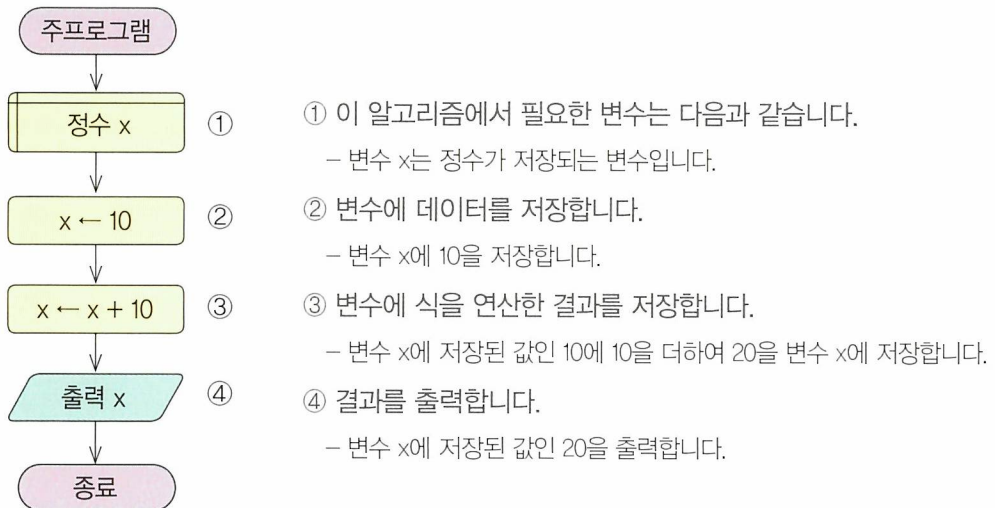
순차 구조를 사용하여 알고리즘을 작성한 순서도의 예는 다음과 같습니다.



순서도 예 독립된 명령인 경우

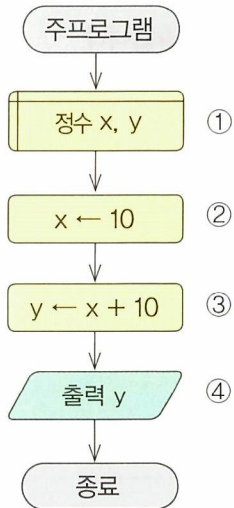


순서도 예 누적 연산인 경우 - 변수에 연산한 결과를 다시 저장하는 경우





순서도 예 명령을 수행한 결과가 다음 명령에 영향을 끼치는 경우



① 이 알고리즘에서 필요한 변수는 다음과 같습니다.

– 변수 x, y는 정수가 저장되는 변수들입니다.

② 변수에 데이터를 저장합니다.

– 변수 x에 10을 저장합니다.

③ 변수에 식을 연산한 결과를 저장합니다.

– 변수 x에 저장된 값인 10에 10을 더하여 20을 변수 y에 저장합니다.

④ 결과를 출력합니다.

– 변수 y에 저장된 값인 20을 출력합니다.

문제 해결을 위한 알고리즘 연습



문제 입력 받은 금액을 500원짜리 동전과 100원짜리 동전을 최소한으로 사용하여 지불하고자 할 때 각각 사용된 동전의 개수를 구하여 결과를 출력해 봅시다.

🕒 문제 해결 방법

입력 받은 금액을 500으로 나누면 몫은 500원짜리 동전의 개수가 되고 나머지를 100으로 나누었을 때의 몫이 100원짜리 동전의 개수가 됩니다.

[입력] 금액 (100원 단위)

[출력] 500원 짜리 동전 개수, 100원 짜리 동전 개수

표에 정리된 예를 살펴봅시다.

입력 금액	500원 동전 개수	100원 동전 개수
1,600	3	1
21,300	42	3
700	1	2

다음과 같은 순서로 문제를 해결할 수 있습니다.

1. 금액을 입력 받습니다.
2. 금액을 500으로 나누어 동전의 개수를 구합니다.
3. 금액을 100으로 나누어 동전의 개수를 구합니다.
4. 500원과 100원 짜리 동전의 개수를 출력합니다.



문제 해결의 예

예 1 | 금액이 1,600원일 경우

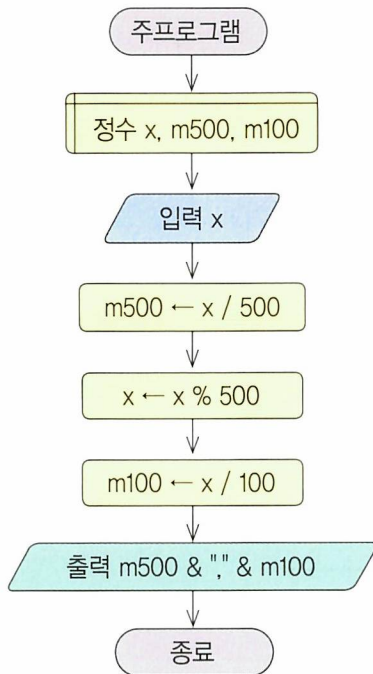


예 2 | 금액이 21,300원일 경우





순서도와 알고리즘 해설



- ① 이 알고리즘에서 필요한 변수는 다음과 같습니다.
 - 변수 x는 금액(정수)을 저장하기 위한 변수입니다.
 - 변수 m500, m100은 각각 500원 짜리와 100원 짜리 동전의 개수를 저장하기 위한 변수입니다.
- ② 변수에 데이터를 저장합니다.
 - 금액(정수)을 입력 받아 변수 x에 저장합니다.
- ③ 변수에 식을 연산한 결과를 저장합니다.
 - 변수 x에 저장된 값을 500으로 나눈 몫을 변수 m500에 저장합니다.
- ④ 변수에 식을 연산한 결과를 저장합니다.
 - 변수 x에 저장된 값을 500으로 나눈 나머지를 변수 x에 저장합니다.
- ⑤ 변수에 식을 연산한 결과를 저장합니다.
 - 변수 x에 저장된 값을 100으로 나눈 몫을 m100에 저장합니다.
- ⑥ 결과를 출력합니다.
 - 변수 m500과 m100에 저장된 값을 출력합니다.



문제 2개의 변수에 저장되어 있는 숫자를 서로 바꾸어 봅시다.

 문제 해결 방법



위와 같이 n1에 저장된 값은 n2로, n2에 저장된 값은 n1으로 숫자를 서로 바꾸려고 할 때, 문제 해결의 [예 1]과 같이 주어진 2개의 변수(n1, n2)만으로는 각 변수에 저장된 값을 서로 바꿀 수 없습니다. 따라서 이 문제를 해결하기 위해서는 2개의 변수 외에 임시로 값을 저장할 변수 즉, 임시 변수를 만들어 문제 해결의 [예 2]와 같이 활용해야 합니다.

[입력] 숫자 2개

[출력] 저장 위치가 바뀐 n1, n2

다음과 같은 순서로 문제를 해결할 수 있습니다.

1. 변수 n1과 n2의 값을 입력 받습니다.
2. 임시 변수 t에 변수 n1의 값을 저장합니다.
3. 변수 n1에 변수 n2의 값을 저장합니다.
4. 변수 n2에 임시 변수 t의 값을 저장합니다.
5. 변수 n1과 n2의 값을 출력합니다.



문제 해결의 예

예 1 변수 2개만 사용한 경우

(1) 경우 1: 먼저 n1의 값을 n2에 저장하고, n2의 값을 n1에 저장하는 경우

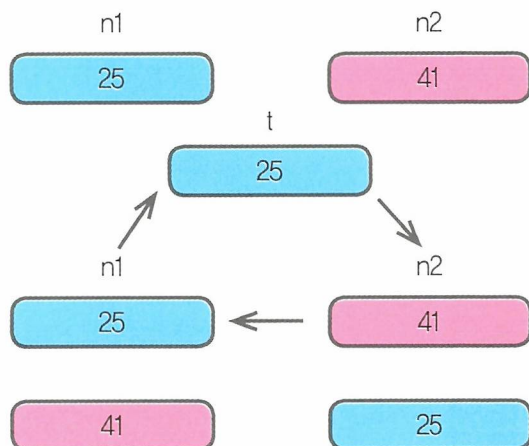


(2) 경우 2: 먼저 n2의 값을 n1에 저장하고, n1의 값을 n2에 저장하는 경우



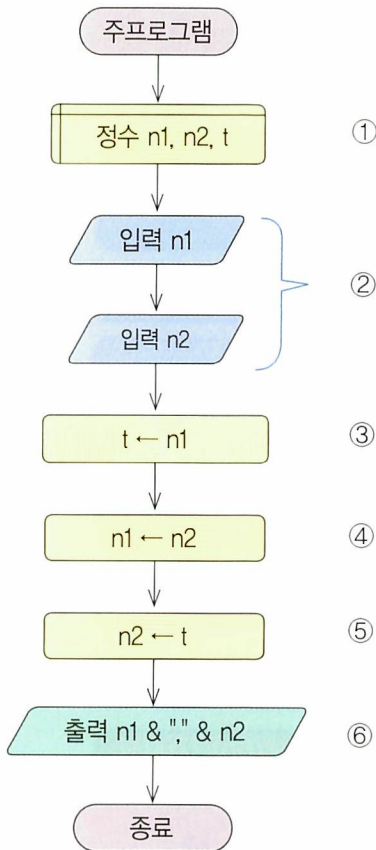
예 2 임시 변수 t를 사용할 경우

- ① 변수 n1과 n2에 값이 저장된 상태
- ↓
- ② 변수 n1의 값을 변수 t에 저장
- ③ 변수 n2의 값을 변수 n1에 저장
- ④ 변수 t의 값을 변수 n2에 저장
- ↓
- ⑤ 결과 출력





순서도와 알고리즘 해설



① 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 n1, n2는 각각 정수를 저장하기 위한 변수들입니다.
- 변수 t는 두 수를 바꿀 때 사용하기 위한 임시 변수입니다.

② 변수에 데이터를 저장합니다.

- 정수를 입력 받아 각각 변수 n1과 n2에 저장합니다 .

③ 변수 t에 변수 n1의 값을 저장합니다.

- 변수 n1의 값을 변수 n2에 저장하기 위하여 임시로 변수 t에 저장합니다. (※ [예 2]의 ⑥ 참고)

④ 변수 n1에 변수 n2의 값을 저장합니다.

- 변수 n2의 값을 변수 n1에 저장합니다. (※ [예 2]의 ㉠ 참고)

⑤ 변수 n2에 변수 t의 값을 저장합니다.

- 변수 t의 값을 변수 n2에 저장합니다. (※ [예 2]의 ㉡ 참고)

⑥ 결과를 출력합니다.

- 변수 n1과 n2에 저장된 값을 출력합니다.



문제 2개의 숫자를 산술 연산한 후, 연산 결과가 다음 연산의 피연산자가 되는 명령들을 수행해 봅시다.

🕒 문제 해결 방법

2개의 숫자를 산술 연산하여 2 결과를 피연산자로 옮겨서 연속된 연산을 수행하기 위해서는 피연산자의 값을 연속적으로 이동시켜야 합니다. 즉, 두 번째 피연산자의 값은 첫 번째 피연산자로, 결과는 두 번째 피연산자로 이동시켜야 연속적으로 연산을 수행할 수 있습니다.

다음의 예를 통해 내용을 이해해 봅시다.

	피연산자 (n1)	피연산자 (n2)	결과 (n3 = n1 × n2)
연산 수행	2	3	6
값의 이동	3	6	

[입력] 숫자 2개

[출력] 피연산자를 순환하여 산술 연산한 결과인 n1, n2

다음과 같은 순서로 문제를 해결할 수 있습니다.

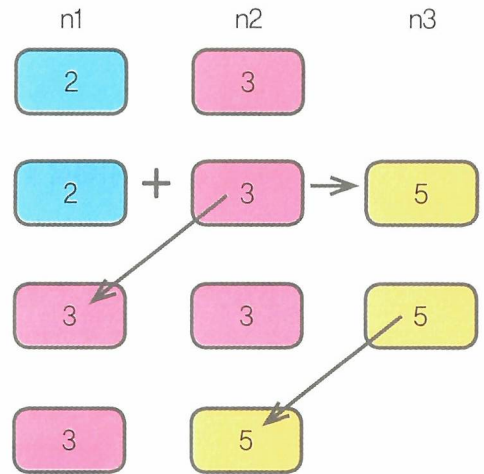
1. 변수 n1과 n2의 값을 입력 받습니다.
2. 변수 n3에 변수 n1과 n2를 연산한 결과를 n3에 저장합니다.
3. 변수 n1에 변수 n2의 값을 저장합니다.
4. 변수 n2에 변수 n3의 값을 저장합니다.



문제 해결의 예

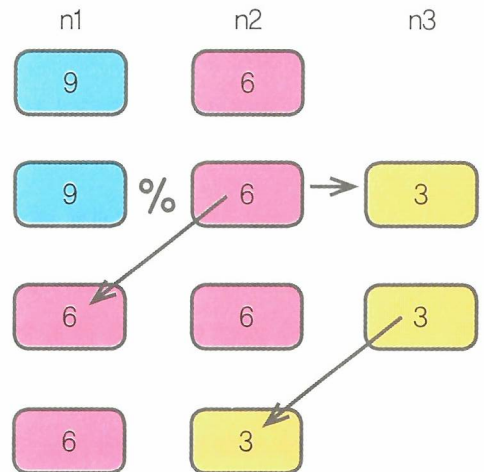
예 1 덧셈 연산(+)을 할 경우

- ㉠ 변수 n1과 n2에 값이 저장된 상태
- ↓
- ㉡ 변수 n1과 n2를 더한 결과를 변수 n3에 저장
- ↓
- ㉢ 변수 n2의 값을 변수 n1에 저장
- ↓
- ㉣ 변수 n3의 값을 변수 n2에 저장



예 2 나머지 연산(%)을 할 경우

- ㉠ 변수 n1과 n2에 값이 저장된 상태
- ↓
- ㉡ 변수 n1을 n2로 나머지 연산한 결과를 변수 n3에 저장
- ↓
- ㉢ 변수 n2의 값을 변수 n1에 저장
- ↓
- ㉣ 변수 n3의 값을 변수 n2에 저장





단순 조건

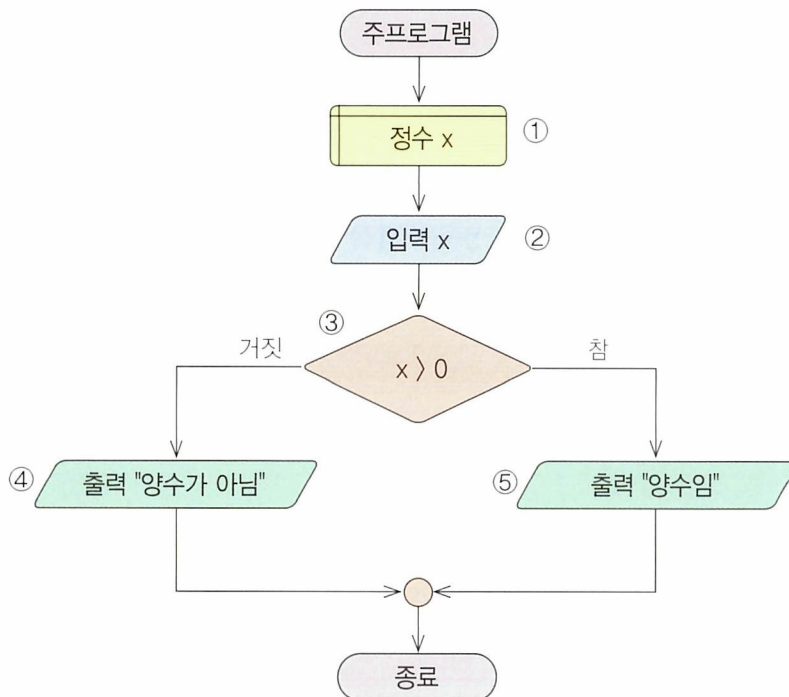
알고리즘이 일반성을 가지기 위해서는 상황에 따라 다르게 동작할 수 있어야 합니다. 알고리즘에서 상황에 따른 판단은 조건으로 구현할 수 있습니다. 조건은 일반적으로 데이터들을 비교하여 참 혹은 거짓으로 판단할 수 있는데 이는 관계 연산자를 사용하여 구현할 수 있습니다.

관계 연산자에는 일반적으로 $=$, $<$, $>$, $<>$, $<=$, $>=$ 등의 연산자 등이 있으며 각 연산자의 의미와 결과는 [표 3-1]과 같습니다.

연산자	의미	예	
		관계식	결과
$=$ (혹은 $==$)	2개의 숫자가 같다.	$3 = 3$	참
		$3 = 7$	거짓
$<$	왼쪽의 숫자가 오른쪽의 숫자보다 작다.	$3 < 7$	참
		$7 < 3$	거짓
$>$	왼쪽의 숫자가 오른쪽의 숫자보다 크다.	$3 > 7$	거짓
		$3 > 7$	참
$<>$ (혹은 $!=$)	2개의 숫자가 다르다.	$3 <> 3$	거짓
		$3 <> 7$	참
$<=$	왼쪽의 숫자가 오른쪽의 숫자보다 작거나 같다.	$3 <= 3$	참
		$3 <= 7$	참
$>=$	왼쪽의 숫자가 오른쪽의 숫자보다 크거나 같다.	$3 >= 3$	참
		$3 >= 7$	거짓

[표 3-1] 관계 연산자

순서도 예



- ① 이 알고리즘에서 필요한 변수는 다음과 같습니다.
 - 변수 x는 정수를 저장하기 위한 변수입니다.
- ② 데이터를 입력 받아 변수에 저장합니다.
 - 정수를 입력 받아 변수 x에 저장합니다.
- ③ 변수 x의 값과 0을 비교합니다.
 - 변수 x의 값이 0보다 크면 참, 그렇지 않으면 거짓이 됩니다.
- ④ 결과를 출력합니다.
 - '거짓'이라면 "양수가 아님" 메시지를 출력합니다.
- ⑤ 결과를 출력합니다.
 - '참'이라면 "양수임" 메시지를 출력합니다.



복합 조건

복합 조건이란 “x가 5보다 작거나 y가 10보다 큰가?”와 같이 여러 조건을 판단하는 것을 말합니다. 이러한 복합 조건은 논리 연산자를 사용하여 만들 수 있습니다. 프로그래밍 언어에서 자주 사용되는 논리 연산자는 다음과 같습니다.

논리곱 (And, 그리고)

논리곱은 피연산자가 모두 참인 경우 결과가 참이 됩니다. [표 3-2]의 예를 살펴봅시다.

A	B	A And B
참	참	참
참	거짓	거짓
거짓	참	거짓
거짓	거짓	거짓

[표 3-2] 논리곱

논리합 (Or, 혹은)

논리합은 피연산자가 모두 거짓인 경우 결과가 거짓이 됩니다. [표 3-3]의 예를 살펴봅시다.

A	B	A Or B
참	참	참
참	거짓	참
거짓	참	참
거짓	거짓	거짓

[표 3-3] 논리합

논리부정 (Not, 아니다)

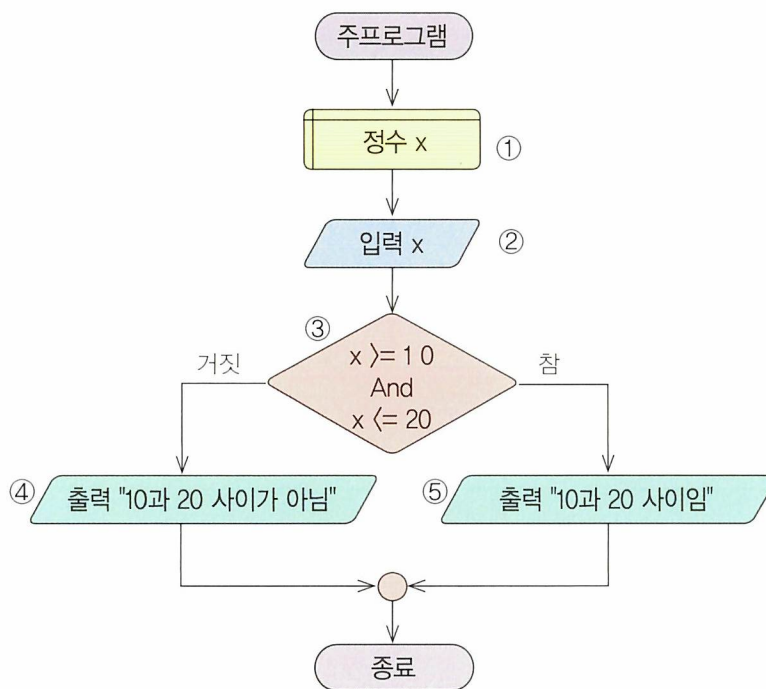
논리부정은 피연산자 값의 반대가 됩니다. 즉, 참이면 거짓, 거짓이면 참이 됩니다. [표 3-4]의 예를 살펴봅시다.



A	Not A
참	거짓
거짓	참

[표 3-4] 논리부정

순서도 예



① 이 알고리즘에서 필요한 변수는 다음과 같습니다. ④ 결과를 출력합니다.

– 변수 x는 정수를 저장하기 위한 변수입니다.

– ‘거짓’이라면 “10과 20 사이가 아님” 메시지를 출력합니다.

② 데이터를 입력 받아 변수에 저장합니다.

– 정수를 입력 받아 변수 x에 저장합니다.

⑤ 결과를 출력합니다.

– ‘참’이라면 “10과 20 사이임” 메시지를 출력합니다.

③ 복합 조건을 위한 논리곱 연산을 합니다.

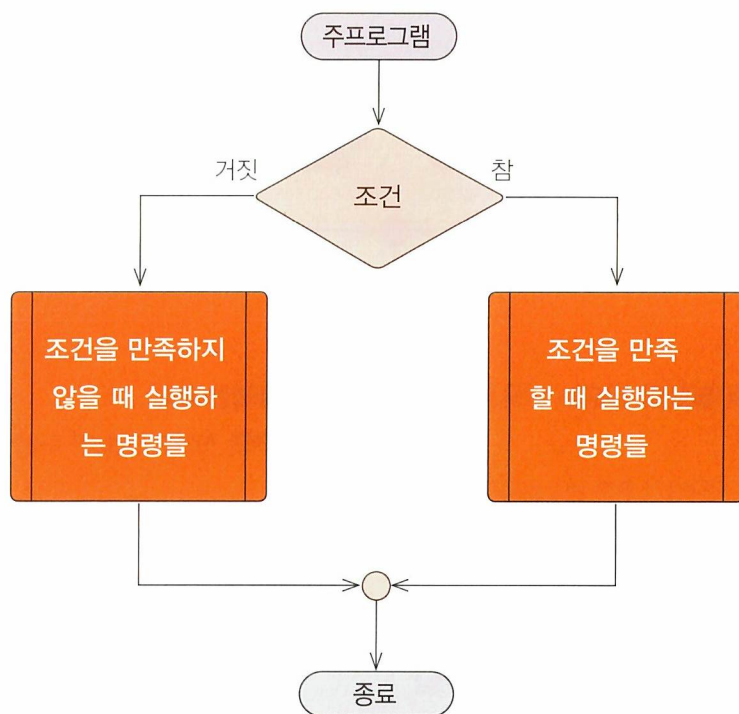
– 변수 x의 값이 10보다 크거나 같고 20보다 작거나 같으면 참, 그렇지 않으면 거짓이 됩니다.



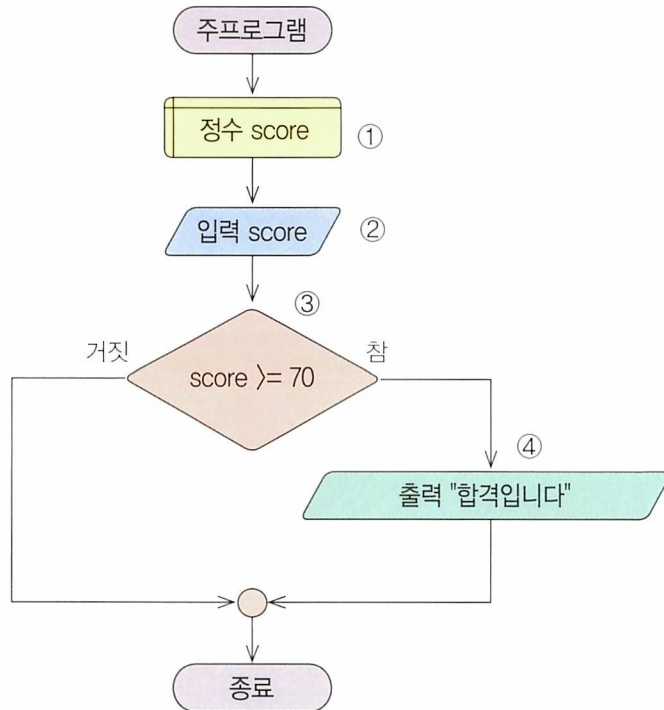
선택 구조

구조화 프로그램의 3개 구조 중의 하나는 선택 구조입니다. 선택 구조는 조건을 검사하여 조건의 결과가 참이냐 거짓이냐에 따라 서로 다른 명령을 수행하는 구조를 말합니다. 그 조건은 단순 조건이나 복합 조건이 올 수 있습니다. 이러한 선택 구조를 활용하여 알고리즘이 상황에 따라 다른 명령을 실행할 수 있습니다. 선택 구조는 조건이 참인 경우에만 명령을 실행할 수도 있고, 참 혹은 거짓에 따라 서로 다른 명령을 실행할 수도 있습니다.

선택 구조를 사용하여 알고리즘을 작성한 순서도의 예는 다음과 같습니다.



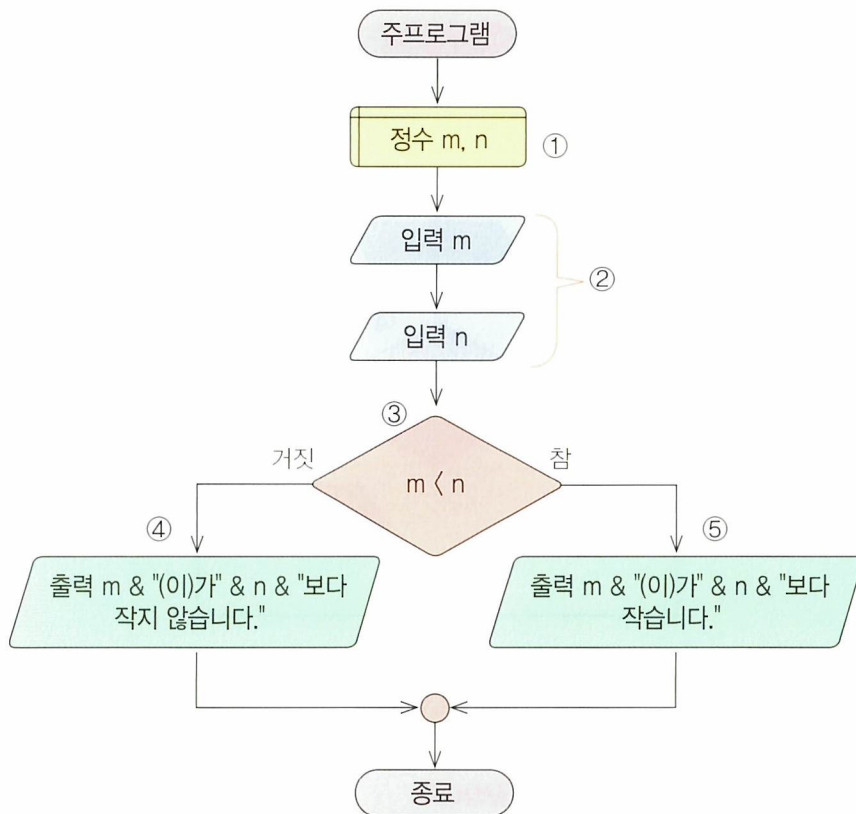
순서도 예 참인 경우에만 실행하는 선택 구조



- ① 이 알고리즘에서 필요한 변수는 다음과 같습니다.
 - 변수 score는 점수를 저장하기 위한 변수입니다.
- ② 데이터를 입력 받아 변수에 저장합니다.
 - 점수를 입력 받아 변수 score에 저장합니다.
- ③ 변수 score의 값과 70을 비교합니다.
 - 변수 score의 값이 70 이상이면 참, 그렇지 않으면 거짓이 됩니다.
- ④ 결과를 출력합니다.
 - '참'이라면 "합격입니다" 메시지를 출력합니다('거짓'이라면 아무 것도 출력되지 않습니다.).

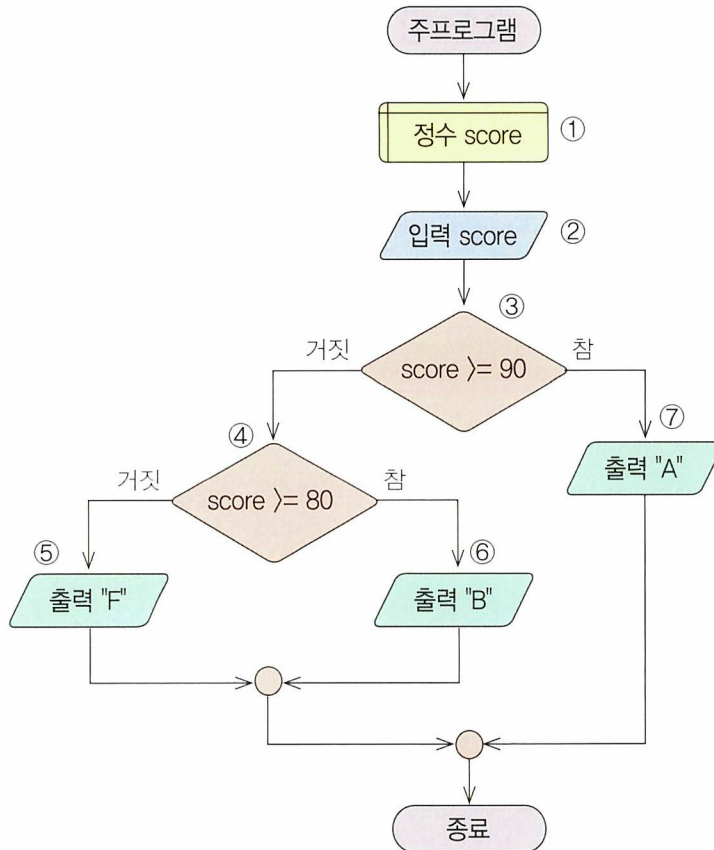


순서도 예 참, 거짓에 따라 다른 명령을 실행하는 선택 구조



- ① 이 알고리즘에서 필요한 변수는 다음과 같습니다.
 - 변수 m, n은 정수를 저장하기 위한 변수들입니다.
- ② 데이터를 입력 받아 변수에 저장합니다.
 - 데이터를 읽어서 변수 m과 n에 저장합니다.
- ③ 변수 m과 변수 n의 크기를 비교합니다.
 - 변수 m의 값이 변수 n의 값보다 작으면 참, 그렇지 않으면 거짓이 됩니다.
- ④ 결과를 출력합니다.
 - '거짓'이라면 "변수 m의 값이 변수 n의 값보다 작지 않습니다." 메시지를 출력합니다.
- ⑤ 결과를 출력합니다.
 - '참'이라면 "변수 m의 값이 변수 n의 값보다 작습니다." 메시지를 출력합니다.

순서도 예 여러 단계로 이루어진 선택 구조



- ① 이 알고리즘에서 필요한 변수는 다음과 같습니다.
- 변수 score는 점수를 저장하기 위한 변수입니다.
- ② 데이터를 입력 받아 변수에 저장합니다.
- 점수를 입력 받아 변수 score에 저장합니다.
- ③ 변수 score의 값과 90을 비교합니다.
- 변수 score의 값이 90 이상이면 참, 그렇지 않으면 거짓이 됩니다.

- ④ 변수 score의 값과 80을 비교합니다.
- 변수 score의 값이 80 이상이면 참, 그렇지 않으면 거짓이 됩니다.
- ⑤ ⑥ 결과를 출력합니다.
- '참'이라면 "B"를, '거짓'이라면 "F"를 출력합니다.
- ⑦ 결과를 출력합니다.
- 변수 score의 값이 90 이상이라면 "A"를 출력합니다.



문제 해결을 위한 알고리즘 연습

문제 첫 번째 입력된 숫자가 두 번째 입력된 숫자로 나누어떨어지는가를 판별하여 출력해 봅시다.

🕒 문제 해결 방법

2개의 숫자를 차례로 입력 받아 첫 번째 숫자를 두 번째 숫자로 나누어, 나머지가 0이면 나누어떨어지는 것이고 0이 아니라면 나누어떨어지지 않는 것입니다.

[입력] 숫자 2개

[출력] 나누어 떨어진다 / 나누어떨어지지 않는다

표에 정리된 예를 살펴봅시다.

첫 번째 숫자	두 번째 숫자	나누어 떨어지는지 여부
15	7	아니오
903	43	예
673	34	아니오

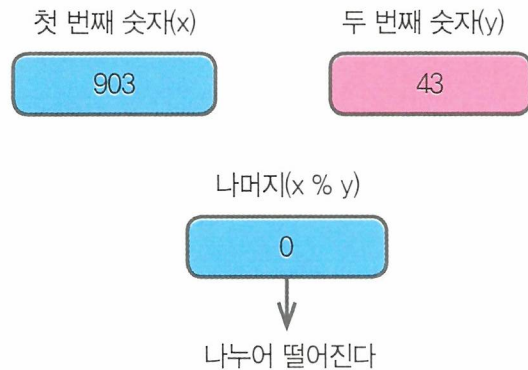
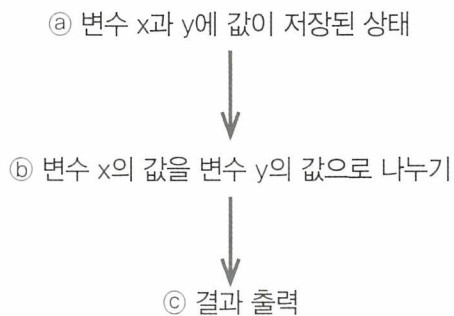
다음과 같은 순서로 문제를 해결할 수 있습니다.

1. 2개의 숫자를 차례로 입력 받습니다.
2. 첫 번째 숫자를 두 번째 숫자로 나눈 후, 나머지를 0과 비교합니다.
 - 2.1 만약 나머지가 0이라면
 - 2.1.1 “나누어떨어진다”라고 출력합니다.
 - 2.2 그렇지 않으면
 - 2.2.1 “나누어떨어지지 않는다”라고 출력합니다.

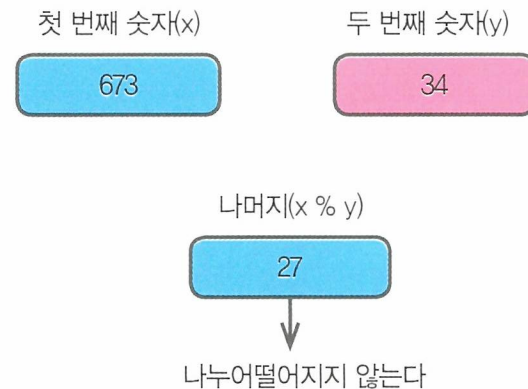
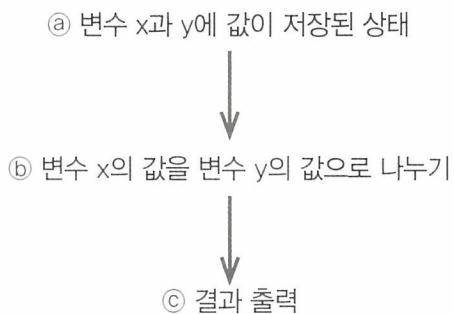


문제 해결의 예

예 1 903, 43을 사례로 입력한 경우

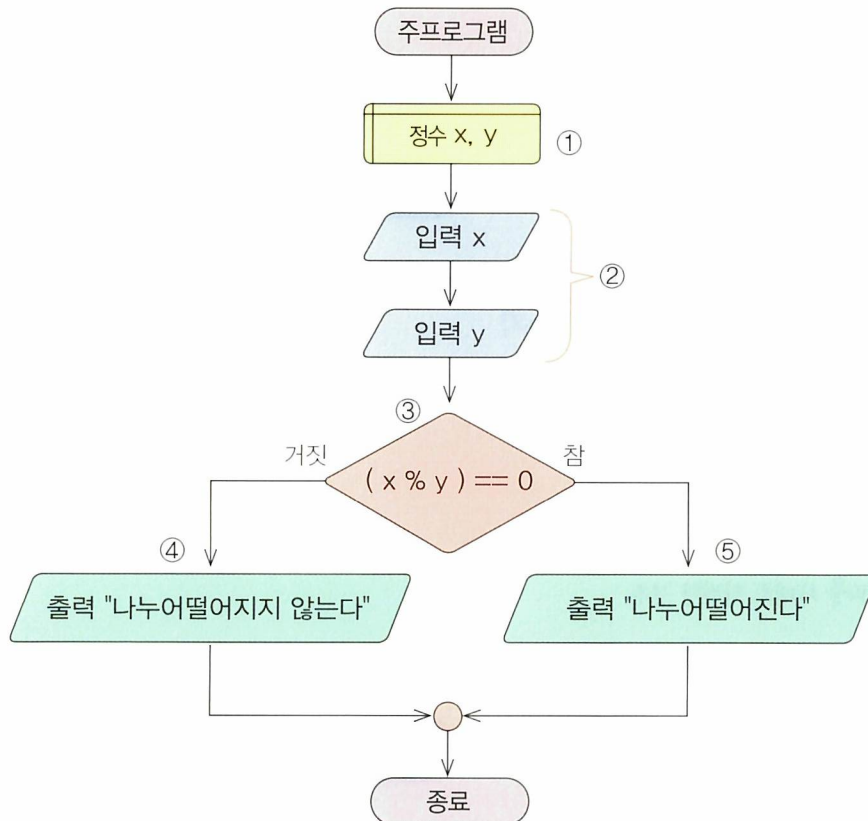


예 2 673, 34를 사례로 입력한 경우





순서도와 알고리즘 해설



① 이 알고리즘에서 필요한 변수는 다음과 같습니다.

– 변수 x, y는 각각 첫 번째 숫자와 두 번째 숫자를 저장하기 위한 변수입니다.

② 데이터를 입력 받아 변수에 저장합니다.

– 첫 번째 숫자를 입력 받아 변수 x에 저장하고, 두 번째 숫자를 입력 받아 변수 y에 저장합니다.

③ 변수 x를 변수 y로 나눈 나머지가 0인지를 검사합니다.

– 나머지가 0이면 참, 그렇지 않으면 거짓이 됩니다.

④ 결과를 출력합니다.

– ‘거짓’이라면 “나누어떨어지지 않는다” 메시지를 출력합니다.

⑤ 결과를 출력합니다.

– ‘참’이라면 “나누어떨어진다” 메시지를 출력합니다.



문제 입력 받은 2개의 숫자 중에서 더 작은 숫자를 찾아 출력해 봅시다.

🕒 문제 해결 방법

입력 받은 2개의 숫자를 비교하여 첫 번째로 입력 받은 숫자가 두 번째로 입력 받은 숫자보다 작으면 첫 번째 숫자를 작은 숫자로 저장하고, 그렇지 않으면 두 번째 숫자를 작은 숫자로 저장합니다. 그 다음, 저장된 작은 숫자를 출력합니다.

[입력] 숫자 2개

[출력] 2개의 숫자 중에서 더 작은 수

표에 정리된 예를 살펴봅시다.

첫 번째 숫자	두 번째 숫자	작은 숫자
4	5	4
5	4	4
4	4	4

다음과 같은 순서로 문제를 해결할 수 있습니다.

1. 2개의 숫자를 입력 받습니다.
2. 2개의 숫자를 비교합니다.
 - 2.1 만약 첫 번째 숫자가 두 번째 숫자보다 작다면
 - 2.1.1 첫 번째 숫자를 작은 숫자로 저장합니다.
 - 2.2 그렇지 않다면
 - 2.2.1 두 번째 숫자를 작은 숫자로 저장합니다.
3. 작은 숫자를 출력합니다.



🔒 문제 해결의 예

|예 1| 변수 m에 4, 변수 n에 5가 저장된 경우

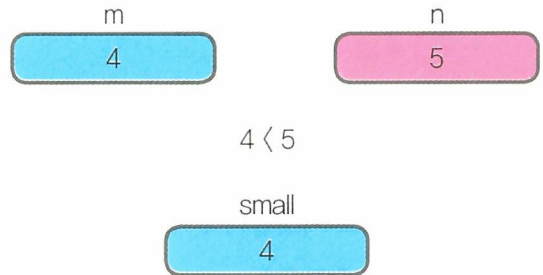
① 변수 m과 n에 값이 저장된 상태



② 변수 m과 변수 n의 값 크기 비교



③ 결과 출력

**|예 2|** 변수 m에 5, 변수 n에 4가 저장된 경우

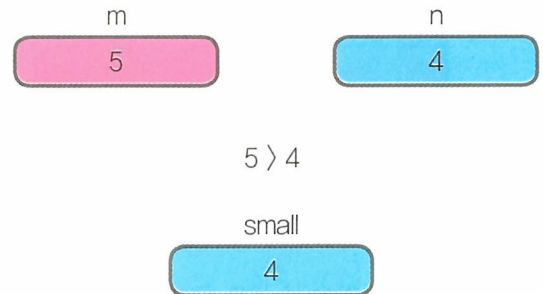
① 변수 m과 n에 값이 저장된 상태



② 변수 m과 변수 n의 값 크기 비교



③ 결과 출력

**|예 3|** 변수 m에 4, 변수 n에 4가 저장된 경우

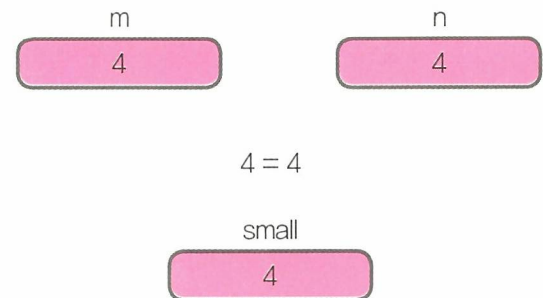
① 변수 m과 n에 값이 저장된 상태



② 변수 m과 변수 n의 값 크기 비교

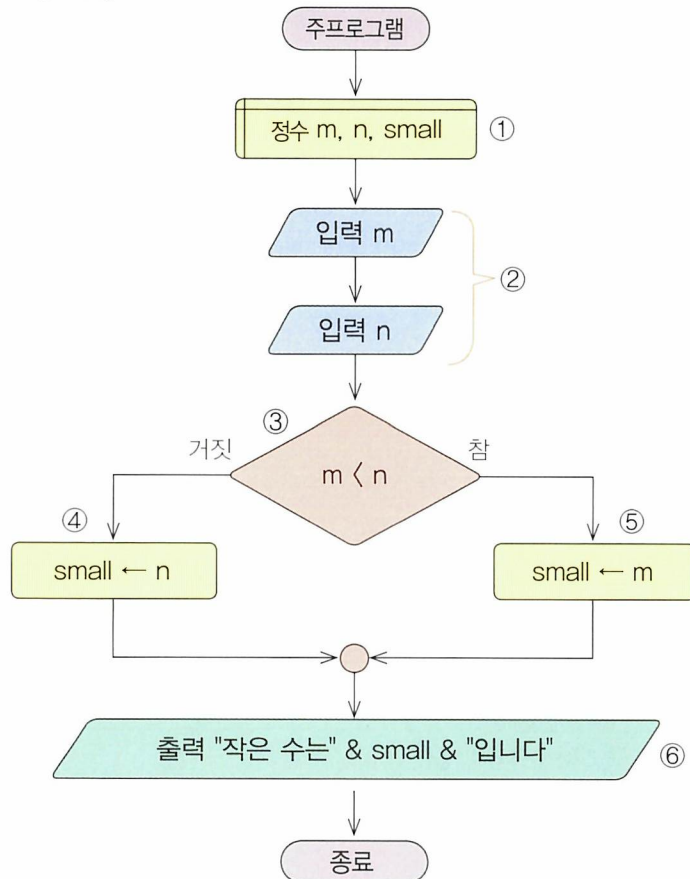


③ 결과 출력





순서도와 알고리즘 해설



① 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 m, n은 입력 받은 정수를 저장하기 위한 변수입니다.
- 변수 small은 작은 숫자를 저장하기 위한 변수입니다.

② 데이터를 입력 받아 변수에 저장합니다.

- 정수를 입력 받아 변수 m과 n에 저장합니다.

③ 변수 m과 변수 n을 비교합니다.

- 변수 m의 값이 변수 n의 값보다 작다면 참, 그렇지 않으면 거짓이 됩니다.

④ '거짓'이라면 변수 small에 변수 n의 값을 저장합니다.

⑤ '참'이라면 변수 small에 변수 m의 값을 저장합니다.

⑥ 결과를 출력합니다.

- "작은 수는 4입니다"(예를 들어, m이 5, n이 4인 경우)라는 메시지를 출력합니다.



문제 2개의 숫자를 비교하여 작은 숫자가 앞에, 큰 숫자가 뒤에 오도록 출력해 봅시다.

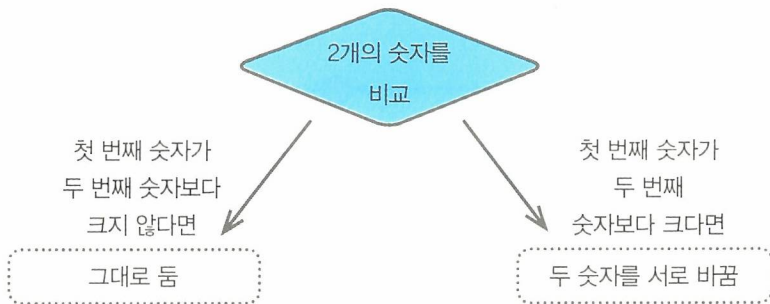
문제 해결 방법

2개의 숫자를 비교하여 만약 첫 번째로 입력 받은 숫자가 두 번째로 입력 받은 숫자보다 크지 않다면 순서대로 되어 있는 것이므로 그대로 출력하고, 첫 번째 숫자가 두 번째 숫자보다 크다면 순서대로 되어 있지 않으므로 두 숫자가 저장되어 있는 위치를 서로 바꾼 후, 출력합니다.

※ 관련 알고리즘 : 2개의 숫자를 서로 바꾸기 (26쪽 참고)

[입력] 숫자 2개

[결과] 크기가 작은 순으로 나열된 2개의 숫자들



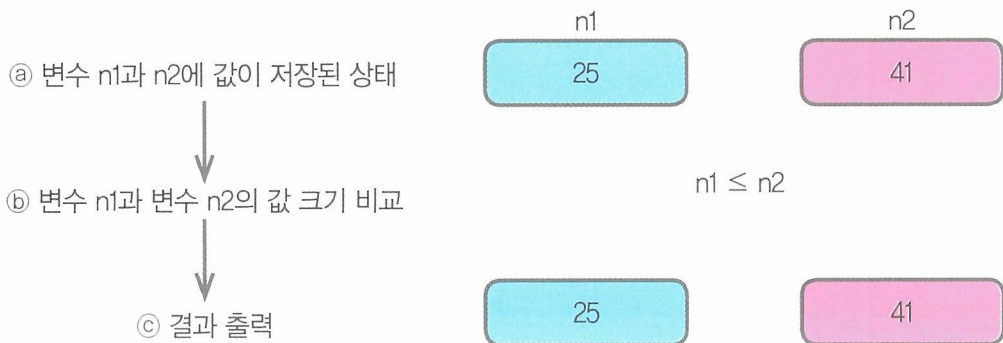
다음과 같은 순서로 문제를 해결할 수 있습니다.

1. 2개의 숫자를 입력 받습니다.
2. 첫 번째로 입력 받은 숫자와 두 번째로 입력 받은 숫자를 비교합니다.
 - 2.1 만약 첫 번째 숫자가 두 번째 숫자보다 크다면
 - 2.1.1 첫 번째 숫자와 두 번째 숫자를 바꿉니다.
3. 첫 번째 숫자와 두 번째 숫자를 출력합니다.



문제 해결의 예

예 1 변수 n1에 25, 변수 n2에 41이 저장되어 있는 경우

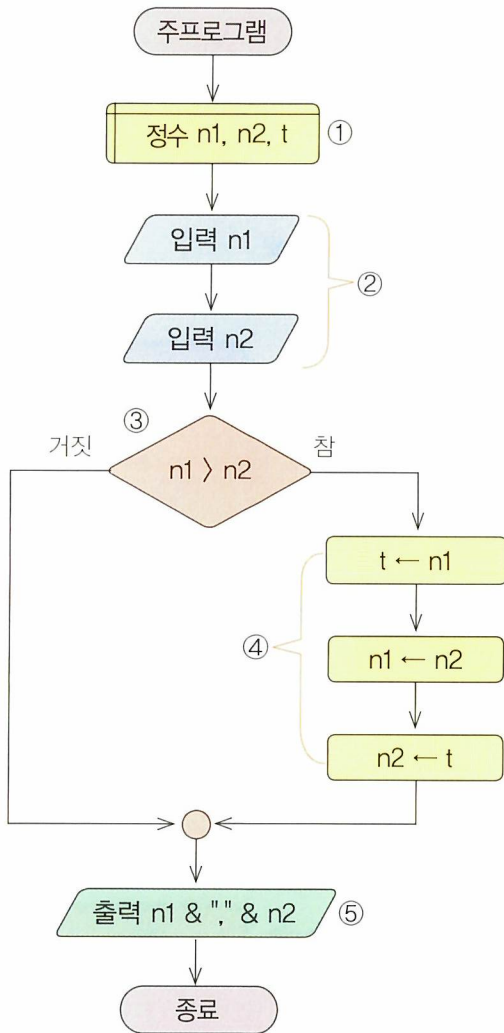


예 2 변수 n1에 89, 변수 n2에 34이 저장되어 있는 경우





순서도와 알고리즘 해설



① 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 n1, n2은 각각 정수를 저장하기 위한 변수들입니다.
- 변수 t는 두 숫자를 바꾸기 위해 사용하는 임시 변수입니다.

② 변수에 데이터를 저장합니다.

- 정수를 입력 받아 각각 변수 n1과 n2에 저장합니다.

③ 변수 n1과 n2의 값을 비교합니다.

- 변수 n1의 값이 변수 n2의 값보다 크면 참, 그렇지 않으면 거짓이 됩니다.

④ '참'이라면 변수 n1의 값을 변수 n2의 값과 서로 바꿉니다.

(※ 26쪽 참고)

⑤ 결과를 출력합니다.

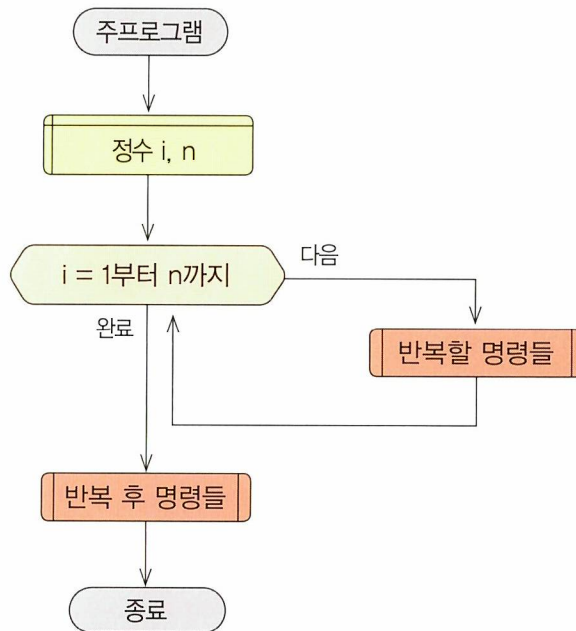
- 변수 n1과 n2에 저장된 값을 출력합니다. 만약 입력된 두 수가 89, 34이라면 "34, 89"로 출력합니다.



반복 구조

구조화 프로그램의 3개 구조 중의 또 다른 하나는 반복 구조입니다. 반복 구조는 명령들을 주어진 조건 혹은 횟수만큼 반복하는 구조를 말합니다. 반복 구조에서 조건은 실행 조건일 수 있고 종료 조건일 수도 있습니다. 실행 조건인 경우에는 조건이 참일 때 명령들을 실행하며, 종료 조건인 경우에는 조건이 참일 때 반복을 끝내고 반복 구조 다음으로 진행합니다. 또한 반복 조건을 반복이 시작되는 시점에 검사할 수도 있고 반복이 끝나는 시점에 조건을 검사할 수도 있습니다. 반복 횟수를 지정하는 경우에는 지정된 횟수만큼 명령을 반복합니다.

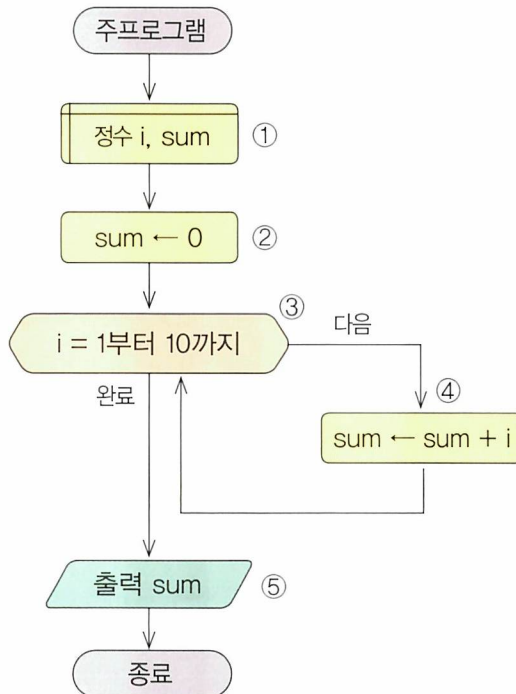
반복 구조를 사용하여 알고리즘을 작성한 순서도의 예는 다음과 같습니다.



이제 1부터 10까지 정수의 합을 구하는 알고리즘을 통해 반복 구조를 살펴보겠습니다. 3가지의 예 모두 반복 구조를 사용하였지만 차이가 있습니다.

순서도 예 · 횟수를 지정한 반복 구조

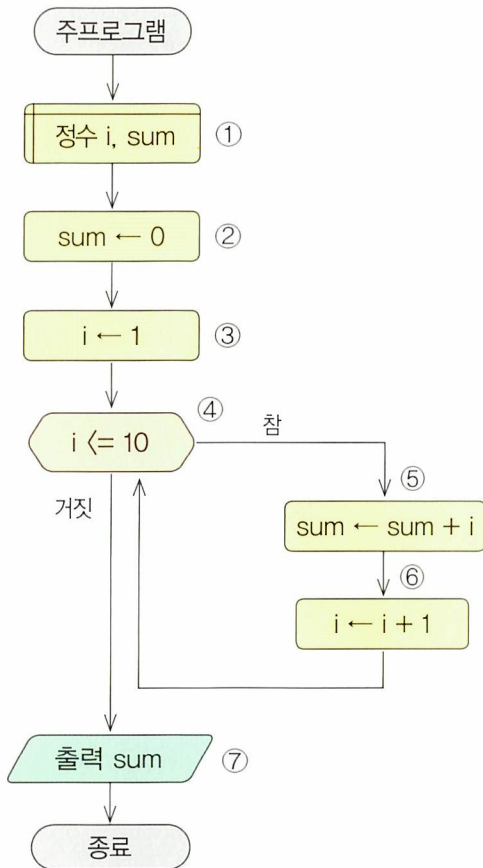
반복 구조의 앞부분에서 반복을 제어하기 위한 초기값, 종료값 및 증감치를 모두 설정하는 경우입니다.



- ① 이 알고리즘에서 필요한 변수는 다음과 같습니다.
 - 변수 i, sum은 정수를 저장하기 위한 변수들입니다.
- ② 합을 저장하기 위한 변수를 초기화합니다.
 - 변수 sum에 0을 저장합니다.
- ③ 반복 횟수를 설정합니다.
 - 변수 i 값을 1부터 10까지 1씩(1인 경우에 생략 가능) 증가시키면서 반복합니다.
- ④ 변수 sum에 변수 i의 값을 누적합니다.
 - 변수 sum의 값에 변수 i의 값을 더하여 변수 sum에 저장합니다.
- ⑤ 결과를 출력합니다.
 - 1부터 10까지의 합이 저장된 변수 sum의 값을 출력합니다.

**순서도 예** 조건을 먼저 검사하는 반복 구조

반복을 수행하기 전에 먼저 조건을 검사하는 경우로 처음부터 조건을 만족하지 않을 때에는 한 번도 수행하지 않을 수 있습니다.



① 이 알고리즘에서 필요한 변수는 다음과 같습니다.

– 변수 i , sum 는 정수를 저장하기 위한 변수들입니다.

② 합을 저장하기 위한 변수를 초기화합니다.

– 변수 sum 에 0을 저장합니다.

③ 반복을 제어하는 변수를 초기화합니다.

– 변수 i 에 1을 저장합니다.

④ 반복 조건을 설정합니다.

– 변수 i 의 값이 10보다 작거나 같은 경우에 명령들을 반복합니다.

⑤ 변수 sum 에 변수 i 의 값을 누적합니다.

– 변수 sum 의 값에 변수 i 의 값을 더하여 변수 sum 에 저장합니다.

⑥ 반복 횟수를 제어하는 변수의 값을 변경합니다.

– 변수 i 의 값을 1만큼 누적합니다.

⑦ 결과를 출력합니다.

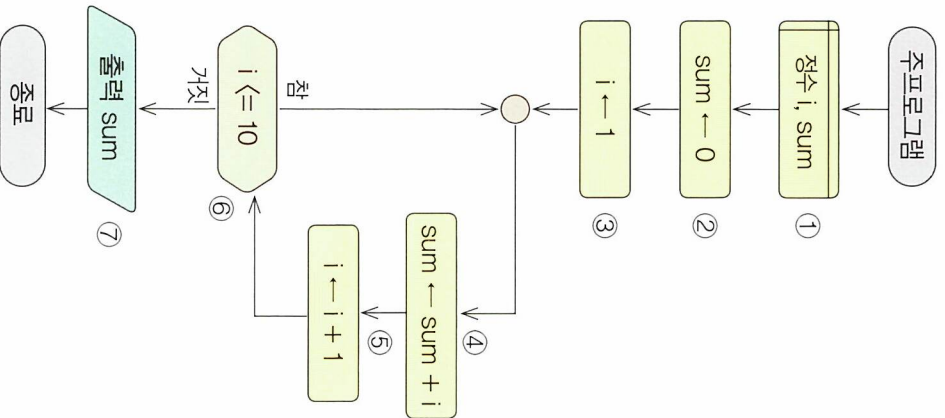
– 변수 sum 의 값을 출력합니다.



반복 구조

순서도에 조건을 나중에 검사하는 반복 구조

반복을 수행한 후에 조건을 검사하는 경우로 반드시 한 번은 수행하게 됩니다.



- ① 이 알고리즘에서 필요한 변수는 다음과 같습니다.
 - 변수 i, sum는 정수를 저장하기 위한 변수들입니다.
- ② 합을 저장하기 위한 변수를 초기화합니다.
 - 변수 sum에 0을 저장합니다.
- ③ 반복을 제어하는 변수를 초기화합니다.
 - 변수 i에 1을 저장합니다.
- ④ 변수 sum에 변수 i의 값을 누적합니다.
 - 변수 sum의 값에 변수 i의 값을 더하여 변수 sum에 저장합니다.
- ⑤ 반복 횟수를 제어하는 변수의 값을 변경합니다.
 - 변수 i의 값을 1만큼 누적합니다.
- ⑥ 반복 조건을 설정합니다.
 - 변수 i의 값이 10보다 작거나 같은 경우에 명령들을 반복합니다.
- ⑦ 결과를 출력합니다.
 - 변수 sum의 값을 출력합니다.